

Peaceman-Rachford Implementation with C++

We seek to solve the following partial differential equation with given interval:

$$v_t = \nu(v_{xx} + v_{yy}), \quad (x, y) \in (0, 1) \times (0, 1), \quad t > 0$$

With boundary conditions:

$$v(0, y, t) = v(1, y, t) = 0, \quad y \in [0, 1], \quad t \geq 0$$

$$v(x, 0, t) = v(x, 1, t) = 0, \quad x \in [0, 1], \quad t \geq 0$$

We use $M_x = M_y = 20$, $\nu = 1.0$, $\Delta t = 0.0005$ and calculate solutions at $t = 0.06$, $t = 0.01$, and $t = 0.9$. We will solve the PDE using the Peaceman-Rachford method and will be compared to the analytical solution, as follows:

$$v(x, y, t) = \sum_{n=1}^{\infty} \sum_{m=1}^{\infty} a_{nm} \sin\left(\frac{n\pi x}{L}\right) \sin\left(\frac{m\pi y}{H}\right) e^{-\lambda_{nm}\nu t}, \quad \text{where } \lambda_{nm} = \left(\frac{n\pi}{L}\right)^2 + \left(\frac{m\pi}{H}\right)^2, \quad a_{nm} = f(x) = \sin(\pi x)$$

The results for the analytical solutions are as follows. Please note that all boundaries are set to 0, and the grid is assumed to be symmetric in both the x- and y-directions.

T = 0.06 Seconds

M_y/M_x	1	2	3	4	5	6	7
1	0.0025	0.0048	0.0066	0.0077	0.0081	0.0077	0.0066
2	0.0049	0.0094	0.0129	0.0152	0.0160	0.0152	0.0129
3	0.0073	0.0138	0.0190	0.0224	0.0235	0.0224	0.0190
4	0.0094	0.0179	0.0246	0.0289	0.0304	0.0289	0.0246
5	0.0113	0.0215	0.0296	0.0348	0.0366	0.0348	0.0296
6	0.0129	0.0246	0.0339	0.0398	0.0419	0.0398	0.0339
7	0.0143	0.0271	0.0373	0.0439	0.0461	0.0439	0.0373
8	0.0152	0.0289	0.0398	0.0468	0.0492	0.0468	0.0398
9	0.0158	0.0301	0.0414	0.0486	0.0511	0.0486	0.0414
10	0.0160	0.0304	0.0419	0.0492	0.0518	0.0492	0.0419

T = 0.1 Seconds

M_y/M_x	1	2	3	4	5	6	7
1	0.0003	0.0007	0.0009	0.0011	0.0011	0.0011	0.0009
2	0.0007	0.0013	0.0018	0.0021	0.0022	0.0021	0.0018
3	0.0010	0.0019	0.0026	0.0031	0.0033	0.0031	0.0026
4	0.0013	0.0025	0.0034	0.0040	0.0042	0.0040	0.0034
5	0.0016	0.0030	0.0041	0.0048	0.0051	0.0048	0.0041
6	0.0018	0.0034	0.0047	0.0055	0.0058	0.0055	0.0047
7	0.0020	0.0038	0.0052	0.0061	0.0064	0.0061	0.0052
8	0.0021	0.0040	0.0055	0.0065	0.0068	0.0065	0.0055
9	0.0022	0.0042	0.0057	0.0068	0.0071	0.0068	0.0057
10	0.0022	0.0042	0.0058	0.0068	0.0072	0.0068	0.0058

T = 0.9 Seconds

M_y/M_x	1	2	3	4	5	6	7
1	2E-21	5E-21	7E-21	8E-21	8E-21	8E-21	7E-21
2	5E-21	9E-21	1E-20	2E-20	2E-20	2E-20	1E-20
3	7E-21	1E-20	2E-20	2E-20	2E-20	2E-20	2E-20
4	9E-21	2E-20	2E-20	3E-20	3E-20	3E-20	2E-20
5	1E-20	2E-20	3E-20	3E-20	4E-20	3E-20	3E-20
6	1E-20	2E-20	3E-20	4E-20	4E-20	4E-20	3E-20
7	1E-20	3E-20	4E-20	4E-20	5E-20	4E-20	4E-20
8	2E-20	3E-20	4E-20	5E-20	5E-20	5E-20	4E-20
9	2E-20	3E-20	4E-20	5E-20	5E-20	5E-20	4E-20
10	2E-20	3E-20	4E-20	5E-20	5E-20	5E-20	4E-20

We define the Peaceman-Rachford method as a numerical approximation for the PDE with the following equations accounting for a full and half step:

$$\frac{u_{j,k}^{n+1/2} - u_{j,k}^n}{\Delta t/2} = \nu \left(\frac{u_{j+1,k}^{n+1/2} - 2u_{j,k}^{n+1/2} + u_{j-1,k}^{n+1/2}}{\Delta x^2} + \frac{u_{j,k+1}^n - 2u_{j,k}^n + u_{j,k-1}^n}{\Delta y^2} \right)$$

$$\frac{u_{j,k}^{n+1} - u_{j,k}^{n+1/2}}{\Delta t/2} = \nu \left(\frac{u_{j+1,k}^{n+1/2} - 2u_{j,k}^{n+1/2} + u_{j-1,k}^{n+1/2}}{\Delta x^2} + \frac{u_{j,k+1}^{n+1} - 2u_{j,k}^{n+1} + u_{j,k-1}^{n+1}}{\Delta y^2} \right)$$

Where, if we define the following:

$$r_x = \frac{\nu \Delta t}{\Delta x^2}, \quad r_y = \frac{\nu \Delta t}{\Delta y^2}$$

The equations above can be simplified as follows:

$$u_{j,k}^{n+1/2} = u_{j,k}^n + \frac{r_x}{2} \left(u_{j+1,k}^{n+1/2} - 2u_{j,k}^{n+1/2} + u_{j-1,k}^{n+1/2} \right) + \frac{r_y}{2} \left(u_{j,k+1}^n - 2u_{j,k}^n + u_{j,k-1}^n \right)$$

$$u_{j,k}^{n+1} = u_{j,k}^{n+1/2} + \frac{r_x}{2} \left(u_{j+1,k}^{n+1/2} - 2u_{j,k}^{n+1/2} + u_{j-1,k}^{n+1/2} \right) + \frac{r_y}{2} \left(u_{j,k+1}^{n+1} - 2u_{j,k}^{n+1} + u_{j,k-1}^{n+1} \right)$$

Using the Peaceman-Rachford method and a timestep of $dt = 0.0005$ s, we gather the following results:

T = 0.06 Seconds

M_y/M_x	1	2	3	4	5	6	7
1	0.0025	0.0048	0.0066	0.0078	0.0082	0.0078	0.0066
2	0.0050	0.0095	0.0131	0.0154	0.0162	0.0154	0.0131
3	0.0073	0.0140	0.0192	0.0226	0.0237	0.0226	0.0192
4	0.0095	0.0181	0.0249	0.0292	0.0307	0.0292	0.0249
5	0.0114	0.0217	0.0299	0.0352	0.0370	0.0352	0.0299
6	0.0131	0.0249	0.0342	0.0402	0.0423	0.0402	0.0342
7	0.0144	0.0274	0.0377	0.0443	0.0466	0.0443	0.0377
8	0.0154	0.0292	0.0402	0.0473	0.0497	0.0473	0.0402
9	0.0160	0.0304	0.0418	0.0491	0.0516	0.0491	0.0418
10	0.0162	0.0307	0.0423	0.0497	0.0523	0.0497	0.0423

T = 0.1 Seconds

M_y/M_x	1	2	3	4	5	6	7
1	0.0004	0.0007	0.0009	0.0011	0.0011	0.0011	0.0009
2	0.0007	0.0013	0.0018	0.0021	0.0023	0.0021	0.0018
3	0.0010	0.0020	0.0027	0.0032	0.0033	0.0032	0.0027
4	0.0013	0.0025	0.0035	0.0041	0.0043	0.0041	0.0035
5	0.0016	0.0030	0.0042	0.0049	0.0052	0.0049	0.0042
6	0.0018	0.0035	0.0048	0.0056	0.0059	0.0056	0.0048
7	0.0020	0.0038	0.0053	0.0062	0.0065	0.0062	0.0053
8	0.0021	0.0041	0.0056	0.0066	0.0070	0.0066	0.0056
9	0.0022	0.0042	0.0058	0.0069	0.0072	0.0069	0.0058
10	0.0023	0.0043	0.0059	0.0070	0.0073	0.0070	0.0059

T = 0.9 Seconds

M_y/M_x	1	2	3	4	5	6	7
1	3E-21	5E-21	8E-21	9E-21	9E-21	9E-21	8E-21
2	6E-21	1E-20	1E-20	2E-20	2E-20	2E-20	1E-20
3	8E-21	2E-20	2E-20	3E-20	3E-20	3E-20	2E-20
4	1E-20	2E-20	3E-20	3E-20	4E-20	3E-20	3E-20
5	1E-20	2E-20	3E-20	4E-20	4E-20	4E-20	3E-20
6	1E-20	3E-20	4E-20	5E-20	5E-20	5E-20	4E-20
7	2E-20	3E-20	4E-20	5E-20	5E-20	5E-20	4E-20
8	2E-20	3E-20	5E-20	5E-20	6E-20	5E-20	5E-20
9	2E-20	3E-20	5E-20	6E-20	6E-20	6E-20	5E-20
10	2E-20	4E-20	5E-20	6E-20	6E-20	6E-20	5E-20

To analyze the accuracy of the Peaceman-Rachford method, the following plots analyze the absolute percentage error between each the analytical and approximate solution at each specific point in time.

Error at T = 0.06 Seconds (%)

M_y/M_x	1	2	3	4	5	6	7
1	1.83	1.83	1.86	1.84	1.85	1.84	1.86
2	1.85	1.85	1.85	1.85	1.85	1.85	1.85
3	1.86	1.84	1.84	1.85	1.85	1.85	1.84
4	1.85	1.85	1.85	1.85	1.85	1.85	1.85
5	1.85	1.85	1.85	1.85	1.85	1.85	1.85
6	1.85	1.85	1.85	1.85	1.85	1.85	1.85
7	1.85	1.85	1.85	1.85	1.85	1.85	1.85
8	1.85	1.85	1.85	1.85	1.85	1.85	1.85
9	1.85	1.85	1.85	1.85	1.85	1.85	1.85
10	1.85	1.85	1.85	1.85	1.85	1.85	1.85

Error at T = 0.1 Seconds (%)

M_y/M_x	1	2	3	4	5	6	7
1	1.55	1.69	1.67	1.64	1.65	1.64	1.67

2	1.61	1.67	1.64	1.63	1.66	1.63	1.64
3	1.64	1.65	1.66	1.65	1.65	1.65	1.66
4	1.67	1.63	1.64	1.65	1.65	1.65	1.64
5	1.68	1.65	1.65	1.63	1.65	1.63	1.65
6	1.64	1.64	1.65	1.64	1.65	1.64	1.65
7	1.65	1.63	1.65	1.65	1.64	1.65	1.65
8	1.63	1.65	1.64	1.65	1.64	1.65	1.64
9	1.65	1.65	1.64	1.64	1.65	1.64	1.64
10	1.66	1.65	1.65	1.64	1.64	1.64	1.65

Error at T = 0.9 Seconds (%)

M_y/M_x	1	2	3	4	5	6	7
1	15.73	15.89	15.89	15.78	15.84	15.78	15.89
2	15.70	15.81	15.53	16.03	15.86	16.03	15.53
3	15.83	16.11	15.85	15.94	15.66	15.94	15.85
4	15.81	15.71	15.70	15.71	15.64	15.71	15.70
5	16.30	15.78	15.99	15.87	15.80	15.87	15.99
6	15.53	15.70	15.78	15.83	15.93	15.83	15.78
7	15.59	15.63	15.83	15.86	15.72	15.86	15.83
8	16.03	15.71	15.83	15.71	15.70	15.71	15.83
9	15.89	15.75	15.90	15.70	15.90	15.70	15.90
10	15.86	15.64	15.93	15.70	15.75	15.70	15.93

Overall, the performance of the Peaceman-Rachford approximation has less than a 2% error before the results approach 0. While not included, other timesteps were analyzed, The Peaceman-Rachford method is capable of increasing the timestep greater than 0.001 seconds while maintaining stability and similar accuracies, while the explicit/explicit methods become unstable when the timestep is larger than 0.0005 seconds.

Works Cited

Thomas, J. (2013). *Numerical partial differential equations: finite difference methods*. Springer Science & Business Media.

Appendix

```
#include <fstream>
#include <vector>
#include <iostream>
#include <Eigen/Dense>
#include <cmath>
#include <iomanip>
#include <string>
using namespace Eigen;

const double nu=1.0;
const double dt=0.0005;
const int Mx=20;
const int My=20;
const double N=1;
```

```

const double dx=1.0 / Mx;
const double dy=1.0 / My;

const double r_x=nu*dt/(dx*dx);
const double r_y=nu*dt/(dy*dy);

const int K = static_cast<int>(N/dt);

MatrixXd A_x=MatrixXd::Zero(Mx-1,Mx-1);
MatrixXd B_x=MatrixXd::Zero(Mx-1,1);
MatrixXd A_y=MatrixXd::Zero(My-1,My-1);
MatrixXd B_y=MatrixXd::Zero(My-1,1);

int K2 =static_cast<int>(2*N/dt+1);
std::vector<MatrixXd> u(K2,MatrixXd::Zero(My+1,Mx+1));

double phi_0(double x, double y){
    return sin(M_PI*x)*sin(2*M_PI*y);
}

Eigen::VectorXd THOM(const Eigen::MatrixXd& A, int m, const Eigen::MatrixXd& phi){
    using namespace Eigen;
    VectorXd alpha =VectorXd::Zero(m);
    VectorXd s= VectorXd::Zero(m);
    VectorXd a(m-1), b(m), c(m-1);

    // Extract diagonals properly
    for (int i = 0; i < m-1; ++i) {
        a(i)= A(i+1,i); // Lower diagonal
        c(i)= A(i,i+1); // Upper diagonal
    }

    for (int i =0; i <m; ++i){
        b(i) =A(i,i); // Main diagonal
    }

    // Flatten phi to get d
    VectorXd d= Eigen::Map<const VectorXd>(phi.data(),phi.size());

    VectorXd phi_2= VectorXd::Zero(m);

    // Forward sweep
    alpha(0)= b(0);
    s(0)= d(0);

    for (int i=1; i< m; ++i){
        alpha(i)=b(i)-a(i-1)*c(i-1)/alpha(i-1);
        s(i) = d(i) - a(i-1)*s(i-1)/alpha(i-1);
    }

    // Backward substitution
    phi_2(m-1)=s(m-1)/alpha(m-1);

```

```

    for (int i=m-2;i>=0; --i) {
        phi_2(i)=(s(i)-c(i)*phi_2(i+1))/alpha(i);
    }
    return phi_2;
}

//Exporting to excel
void write_csv_slice(const std::vector<std::vector<std::vector<double>>>& u, double t, double dt, const std::string& file
name) {
    int timestep = static_cast<int>(t / dt);

    std::ofstream file(filename);
    if (!file) {
        std::cerr << "Could not open file: " << filename << "\n";
        return;
    }

    // Optional header: x0, x1, x2, ...
    for (int j=0; j < u[0].size(); ++j) {
        file << "x" << j;
        if (j < u[0].size() - 1) file << ",";
    }
    file << "\n";

    for (int i=0; i<u.size(); ++i) {
        for (int j=0; j<u[i].size(); ++j) {
            file << std::setprecision(10) << u[i][j][timestep];
            if (j < u[i].size() - 1) file << ",";
        }
        file << "\n";
    }

    file.close();
    std::cout << "Wrote: " << filename << "\n";
}

int main() {
    //Defining A_x and A_y
    for (int i=0; i<Mx-1; ++i) {
        for (int j=0; j<My-1; ++j){
            if (i==j){
                A_x(i,j)=r_x+1;
                A_y(i,j) = r_y+1;
            }
            else if (i==j-1){
                A_x(i,j)=-r_x/2;
                A_y(i,j)=-r_y/2;
            }
            else if (i==j+1){
                A_x(i,j)=-r_x/2;
                A_y(i,j)=-r_y/2;
            }
        }
    }
}

```

```

// Initial conditions
for (int i=1; i<My; ++i){
    for (int j=1; j<Mx; ++j){
        //u[0](i,j)=phi_0(dx*j,dy*i);
        u[0](i,j) = phi_0(dx*i,dy*j);
    }
}

for (int n=1; n<= static_cast<int>(N/dt); ++n){
    // X-direction sweep
    for (int i=1; i<My; ++i){
        for (int j=1; j<Mx; ++j){
            B_x(j-1, 0)=(r_y/2.0)*u[2*n-2](j,i+1)+(1-r_y)*u[2*n-2](j,i)+(r_y/2.0)*u[2*n-2](j,i-1);
        }
        Eigen::VectorXd result = THOM(A_x, Mx-1, B_x);
        for (int j=1; j<Mx; ++j){
            u[2*n-1](i,j)=result(j-1);
        }
    }

    //Enforcing boundary conditions
    for (int i=0; i<=My; ++i){
        u[2*n-1](i,0)=0;// Left boundary
        u[2*n-1](i,Mx)=0;// Right boundary
    }
    for (int j=0; j<=Mx; ++j){
        u[2*n-1](0,j)=0;// Bottom boundary
        u[2*n-1](My,j)=0;// Top boundary
    }

    //Y-direction sweep
    for (int j=1; j<Mx; ++j){
        for (int i=1; i<My; ++i){
            B_y(i-1, 0) = (r_x / 2.0) * u[2 * n - 1](j+1,i) + (1 - r_x) * u[2 * n - 1](j,i) + (r_x / 2.0) * u[2 * n - 1](j-1,i);
        }
        Eigen::VectorXd result = THOM(A_y, My-1, B_y);
        for (int i=1; i<My; ++i){
            u[2*n](i,j)=result(i-1);
        }
    }

    //Enforcing boundary conditions
    for (int i=0; i<=My; ++i){
        u[2*n](i,0)=0;// Left boundary
        u[2*n](i,Mx)=0;// Right boundary
    }
    for (int j=0; j<=Mx; ++j){
        u[2*n](0,j)=0;// Bottom boundary
        u[2*n](My,j)=0;// Top boundary
    }
}

// End of numerical portion

```

```

//Exporting to Excel
int steps = static_cast<int>(2*N/dt+1);

std::vector<std::vector<std::vector<double>>> u_export(My+1, std::vector<std::vector<double>>(Mx+1, std::vector<double>(steps)));
for (int t=0; t<steps; ++t) {
    for (int i=0; i<=My; ++i) {
        for (int j = 0; j<= Mx; ++j) {
            u_export[i][j][t]=u[t](i,j);
        }
    }
}

write_csv_slice(u_export, 2*0.06, dt, "T_0.06.csv");
write_csv_slice(u_export, 2*0.1, dt, "T_0.1.csv");
write_csv_slice(u_export, 2*0.9, dt, "T_0.9.csv");

return 0;
}

```